DOCUMENTO DE DISEÑO DE BAJO NIVEL (LLD)

Transaction Execution API

Ejecución de transacciones de crédito y débito contra un proveedor externo

Autor: Miguel Arturo Quiroz S.

Contexto: Backend Challenge - Spin

Versión: 1.0

Fecha: 22 de octubre de 2026

Control de versiones

Versión	Fecha	Autor	Cambios
1.0	22/10/2026	Miguel Arturo Quiroz	Versión inicial.

Contenido

1. Introducción 4
    1.1 Propósito 4
    1.2 Alcance 4
    1.3 Términos clave 4
2. Requerimientos 5
    2.1 Requerimientos funcionales 5
    2.2 Reglas de negocio 5
    2.3 Atributos de calidad 5
3. Vista general de la solución 7
    3.1 Componentes 7
4. Decisiones de arquitectura (ADR) 9
    ADR-01 — Arquitectura hexagonal ligera 9
    ADR-02 — PostgreSQL como almacenamiento 9
    ADR-03 — Registro previo (write-ahead) y transacciones de base de datos cortas 10
    ADR-04 — Distinguir FAILED de UNKNOWN y reintentar solo cuando es seguro 11
    ADR-05 — Despliegue de demostración en AWS EC2 con docker-compose 11
5. Decisiones de diseño 13
    5.1 Lenguaje y framework: Java 25 y Spring Boot 4.1 13
    5.2 Acceso a datos: SQL explícito con JdbcClient 13
    5.3 Resiliencia ante el proveedor 13
    5.4 Resultado del proveedor como tipo cerrado 14
    5.5 Concurrencia: virtual threads 14
    5.6 Idempotencia del API 14
    5.7 Representación del dinero 15
    5.8 Identificadores: UUIDv7 15
    5.9 Errores y códigos HTTP 15
    5.10 Paginación 16
    5.11 Proveedor simulado con WireMock 16
    5.12 Migraciones 16
6. Flujos principales 17
    6.1 Ejecución de una transacción 17
    6.2 Reintento del cliente (idempotencia) 19
    6.3 Estados de una transacción 20

7. Modelo de datos **21**
7.1 Índices **21**
8. Estrategia de pruebas **22**
9. Despliegue **23**
9.1 Ambiente local **23**
9.2 Ambiente de demostración en AWS **23**
9.3 Topología productiva objetivo **23**
10. Riesgos y mitigaciones **25**
11. Evolución y fuera de alcance **26**
12. Uso de inteligencia artificial **27**

1. Introducción

1.1 Propósito

Este documento describe el diseño de la **Transaction Execution API**: qué componentes la forman, cómo interactúan y las razones detrás de cada decisión técnica. Está dirigido a una audiencia de arquitectura y liderazgo técnico; el detalle de implementación vive en el código y en su README.

1.2 Alcance

El servicio recibe solicitudes de transacciones de crédito y débito, aplica reglas de negocio, las ejecuta contra un proveedor externo que es dueño de los saldos y bloqueos de la cuenta, persiste el resultado y expone una consulta con filtros y paginación.

Dentro del alcance	Fuera del alcance
Ejecución de transacciones CREDIT y DEBIT	Gestión de saldos y cuentas (responsabilidad del proveedor)
Reglas de negocio previas a la ejecución	Conciliación automática de transacciones con resultado incierto
Persistencia y consulta de transacciones	Autenticación y autorización
Resiliencia ante fallas del proveedor	Publicación de eventos a otros servicios
Idempotencia del API	Implementación real del proveedor (se simula)

1.3 Términos clave

Término	Significado en este documento
Proveedor	Sistema externo que valida y aplica el movimiento sobre la cuenta. Se simula con WireMock.
Idempotencia	Propiedad de que repetir la misma solicitud produzca el mismo efecto que hacerla una vez.
Write-ahead	Registrar la intención (estado PENDING) antes de ejecutar la acción con efecto externo.
Circuit breaker	Mecanismo que deja de llamar a una dependencia degradada durante un tiempo para no saturar ni saturarse.
Resultado incierto	Situación en que la petición pudo llegar al proveedor pero no hay una respuesta confiable sobre si se ejecutó o no.

2. Requerimientos

2.1 Requerimientos funcionales

ID	Requerimiento
RF-01	Recibir solicitudes de transacción con cuenta, tipo (CREDIT/DEBIT), monto, moneda y descripción.
RF-02	Validar la solicitud y aplicar las reglas de negocio antes de contactar al proveedor.
RF-03	Ejecutar la transacción contra el proveedor externo.
RF-04	Persistir la transacción con el resultado del proveedor, incluido el saldo resultante.
RF-05	Responder con la transacción creada y su estado.
RF-06	Consultar transacciones con filtros (cuenta, estado, tipo) y paginación.

2.2 Reglas de negocio

ID	Regla	Comportamiento
RN-01	Monto mínimo	El monto debe ser **mayor** a 1.00. Un monto de exactamente 1.00 se rechaza.
RN-02	Tope de débito	Un DEBIT no puede exceder 10,000.00 por operación (10,000.00 sí se permite). CREDIT no tiene tope.
RN-03	Moneda	Solo se acepta MXN.

Si una regla no se cumple, la transacción **no se ejecuta** y se responde con un error descriptivo que lista todas las violaciones encontradas, no solo la primera.

2.3 Atributos de calidad

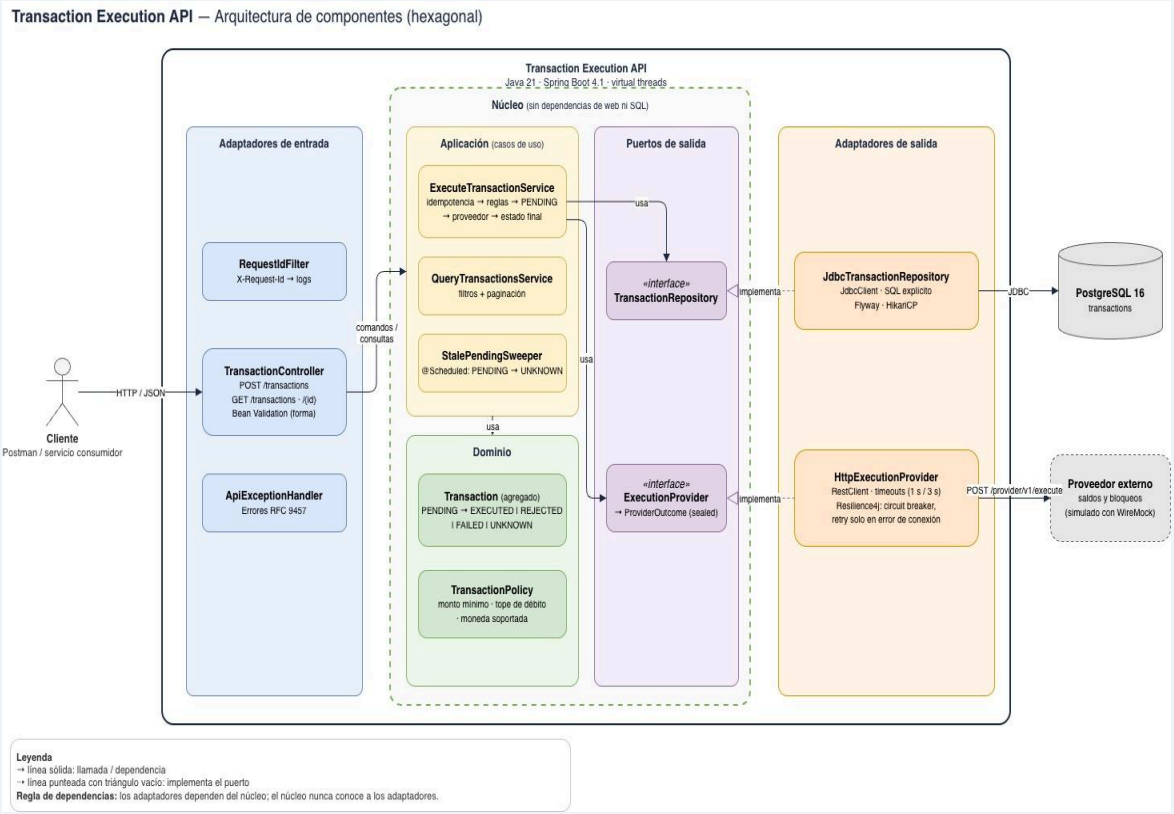
Ordenados por prioridad. Cuando dos entran en conflicto, prevalece el de mayor prioridad.

#	Atributo	Qué exige	Cómo se atiende
1	Correctitud financiera	No ejecutar dos veces una transacción; no reportar como fallida una que pudo ejecutarse.	Write-ahead, estados FAILED/UNKNOWN, idempotencia, reintentos selectivos (ADR-03, ADR-04).
2	Alto volumen	Millones de transacciones diarias, con picos.	Transacciones de BD cortas, virtual threads, índices alineados a las consultas, paginación sin conteo total.
3	Resiliencia	Degradarse con control ante fallas del proveedor.	Timeouts, circuit breaker, clasificación explícita de fallas.
4	Simplicidad	No sobre-ingenierizar.	Un servicio, una base de datos, sin infraestructura que el problema no pide.

#	Atributo	Qué exige	Cómo se atiende
5	Mantenibilidad	Código que comunica intención y se prueba fácilmente.	Arquitectura hexagonal, dominio sin frameworks, SQL explícito.

3. Vista general de la solución

La solución es un único servicio REST con su propia base de datos PostgreSQL, que se comunica con el proveedor externo por HTTP. Internamente sigue una **arquitectura hexagonal**: el núcleo (dominio y casos de uso) no depende de frameworks ni de infraestructura; los adaptadores traducen entre el núcleo y el mundo exterior (HTTP de entrada, base de datos, HTTP del proveedor).



3.1 Componentes

Componente	Capa	Responsabilidad
Controlador REST	Adaptador de entrada	Traduce HTTP a comandos y consultas; válida la forma de la solicitud.
Manejador de errores	Adaptador de entrada	Convierte excepciones en respuestas de error estándar (RFC 9457).
Caso de uso: ejecutar transacción	Aplicación	Orquesta idempotencia, reglas, registr, llamada al proveedor y estado final.

Componente	Capa	Responsabilidad
Caso de uso: consultar transacciones	Aplicación	Consultas con filtros y paginación.
Depurador de pendientes	Aplicación	Tarea periódica que mueve a UNKNOWN las transacciones que quedaron PENDING demasiado tiempo.
Transacción (agregado)	Dominio	Datos de la transacción y transiciones de estado válidas.
Política de reglas	Dominio	Evalúa las reglas de negocio y reporta todas las violaciones.
Puerto de repositorio	Dominio	Contrato de persistencia que el núcleo necesita.
Puerto del proveedor	Dominio	Contrato de ejecución; devuelve un resultado tipado en lugar de lanzar excepciones.
Repositorio JDBC	Adaptador de salida	Implementa la persistencia con SQL explícito sobre PostgreSQL.
Cliente del proveedor	Adaptador de salida	Llamada HTTP con timeouts, circuit breaker y clasificación de fallas.

4. Decisiones de arquitectura (ADR)

El problema tiene pocas decisiones con impacto estructural. Se registran aquí las cinco que condicionan el resto del diseño; las decisiones de menor alcance están en la sección 5.

ADR-01 — Arquitectura hexagonal ligera

Estado: Aceptada

Contexto: El servicio integra tres elementos externos (clientes HTTP, base de datos, proveedor) y el comportamiento más delicado está en cómo reacciona ante las fallas del proveedor. Ese comportamiento debe poder probarse de forma aislada y exhaustiva.

Opciones consideradas:

Opción	A favor	En contra
Capas clásicas (controller → service → repository)	Conocida, poca ceremonia.	El servicio queda acoplado a la tecnología de persistencia y al cliente HTTP; probar fallas del proveedor exige levantar más infraestructura.
Hexagonal en un solo módulo	Núcleo testeable con dobles simples; dependencias apuntan hacia el dominio.	Dos interfaces de puerto extra.
Hexagonal multi-módulo	Separación forzada por el compilador.	Ceremonia de build desproporcionada para un servicio de este tamaño.

Decisión: Arquitectura hexagonal en un solo módulo, con separación por paquetes (`domain`, `application`, `adapter`, `config`).

Justificación: El costo es mínimo (dos interfaces) y el beneficio es directo: todos los escenarios del proveedor (aprobado, rechazado, timeout, caída) se prueban contra el caso de uso con implementaciones en memoria, sin red ni base de datos. Además, cambiar de proveedor o de motor de base de datos solo toca un adaptador.

Consecuencias: La regla de dependencias (el núcleo no importa adaptadores ni frameworks web/SQL) se mantiene por convención y puede reforzarse con una prueba de arquitectura (ArchUnit).

ADR-02 — PostgreSQL como almacenamiento

Estado: Aceptada

Contexto: Se persisten registros financieros que deben ser exactos, únicos por solicitud y consultables por cuenta, estado y fecha, en un contexto de alto volumen.

Opciones consideradas:

Opción	A favor	En contra
En memoria	Cero infraestructura.	No sobrevive reinicios ni permite varias instancias; no representa un escenario productivo.

Opción	A favor	En contra
SQLite	Archivo local, simple.	Escrituras serializadas; no apta para concurrencia alta ni para varias instancias.
MongoDB	Esquema flexible, escala horizontal.	El dato es tabular y estable; se pierden constraints declarativos que aquí son la garantía de correctitud.
PostgreSQL	ACID, constraints UNIQUE y CHECK, NUMERIC exacto, índices compuestos.	Requiere un contenedor o servicio administrado.

Decisión: PostgreSQL 16.

Justificación: Dos garantías del diseño descansan en la base de datos y no en el código: la unicidad de la llave de idempotencia (constraint UNIQUE) y la transición atómica de estado (UPDATE condicional). Además, NUMERIC evita errores de redondeo en montos y los índices compuestos sirven directamente a la consulta principal (por cuenta, ordenada por fecha). Es parte del stack de Spin.

Consecuencias: El esquema se versiona con changelogs de liquibase. Las pruebas de persistencia usan una instancia real en contenedor (Testcontainers) en lugar de una base embebida que se comporta distinto.

ADR-03 — Registro previo (write-ahead) y transacciones de base de datos cortas

Estado: Aceptada

Contexto: Ejecutar una transacción implica un efecto externo (mover dinero en el proveedor) y un registro local. No existe una transacción distribuida entre ambos. Además, la llamada al proveedor puede tardar hasta el timeout configurado.

Opciones consideradas:

Opción	A favor	En contra
Llamar al proveedor y luego guardar	Un solo INSERT.	Si el proceso cae después de ejecutar y antes de guardar, el movimiento de dinero queda invisible para el sistema.
Una transacción de BD que envuelve todo el flujo	Parece atómico.	No lo es (el proveedor no participa); además retiene una conexión del pool durante toda la llamada de red y agota el pool bajo carga.
Registrar PENDING, llamar, actualizar el estado (dos transacciones cortas)	Siempre queda rastro del intento; las conexiones solo se usan unos milisegundos.	Requiere manejar transacciones que se quedan en PENDING.

Decisión: Insertar la transacción en estado PENDING en una transacción corta, llamar al proveedor sin ninguna transacción de base de datos abierta, y actualizar el estado final en una segunda transacción corta protegida por `UPDATE ... WHERE status = 'PENDING'`.

Justificación: Con cientos de solicitudes por segundo y llamadas al proveedor de hasta 3 segundos, mantener una conexión por solicitud durante la llamada agotaría un pool de 20 conexiones con muy poca carga, aunque la base de datos esté ociosa. El registro previo garantiza que ningún intento de mover dinero quede sin rastro.

Consecuencias: Si el proceso cae entre la llamada y la actualización, la transacción queda en PENDING. Un depurador periódico la mueve a UNKNOWN tras un umbral (5 minutos) para que entre a conciliación. El UPDATE condicional evita que dos actores (el flujo normal y el depurador) se pisen.

ADR-04 — Distinguir FAILED de UNKNOWN y reintentar solo cuando es seguro

Estado: Aceptada

Contexto: El contrato del proveedor **no acepta llave de idempotencia**. Si una solicitud de débito llega al proveedor, se ejecuta y la respuesta se pierde (timeout), reenviarla puede cobrar dos veces.

Opciones consideradas:

Opción	A favor	En contra
Un solo estado de error y reintentos automáticos	Simple.	Riesgo real de doble cargo; el cliente también reintentaría ante un error que no lo es.
Un solo estado de error sin reintentos	Sin doble cargo.	Mezcla 'no se ejecutó' con 'no sé si se ejecutó'; el cliente no puede decidir si reintentar.
FAILED (seguro que no se ejecutó) vs UNKNOWN (pudo ejecutarse), reintento solo en errores de conexión	Cada estado dice algo verdadero; el reintento solo ocurre cuando la petición no salió.	Requiere clasificar cada tipo de falla con cuidado.

Decisión: Clasificar cada respuesta o falla del proveedor en uno de cuatro resultados (aprobado, rechazado, no ejecutado, incierto) y reintentar **una sola vez** únicamente cuando la conexión no se pudo establecer.

Justificación: En un sistema de pagos, reportar como fallida una transacción que sí se ejecutó es peor que reportarla como incierta: lo primero invita a repetir el cargo, lo segundo invita a verificar. Un error de conexión garantiza que la petición no llegó, por eso es el único caso en que reintentar es seguro.

Consecuencias: Las transacciones UNKNOWN requieren conciliación (fuera del alcance, documentada como evolución). Si el proveedor aceptara una llave de idempotencia en el futuro, los reintentos se podrían ampliar a timeouts.

ADR-05 — Despliegue de demostración en AWS EC2 con docker-compose

Estado: Aceptada

Contexto: Se quiere dejar el servicio funcionando en un ambiente público para que los revisores lo prueben, con bajo costo y sin traducir el empaquetado a otra plataforma.

Opciones consideradas:

Opción	A favor	En contra
EKS (Kubernetes)	Igual al entorno de Spin.	~US$73/mes solo por el plano de control; configuración desproporcionada para un demo.
ECS Fargate + RDS	Arquitectura de producción.	Más piezas y más costo para un ambiente temporal.
EC2 con docker-compose	El mismo artefacto que corre local; ~US$12/mes; se levanta en minutos.	Una sola instancia, base de datos en contenedor: no es un ambiente productivo.

Decisión: Una instancia EC2 (t4g.small, Amazon Linux 2023) ejecutando la API, PostgreSQL y el proveedor simulado con docker-compose. Solo el puerto de la API queda expuesto.

Justificación: El objetivo del ambiente es demostrar el comportamiento, no la operación productiva. Reutilizar exactamente el docker-compose local elimina diferencias entre lo que se prueba y lo que se despliega.

Consecuencias: El ambiente no tiene alta disponibilidad ni autenticación y se apaga al terminar el proceso. La sección 9.3 describe la topología productiva objetivo.

5. Decisiones de diseño

Decisiones de menor alcance que el ADR, pero que un revisor puede cuestionar. Cada una indica qué se eligió, por qué y qué se sacrifica.

5.1 Lenguaje y framework: Java 25 y Spring Boot 4.1

Decisión: Java 25 (LTS) con Spring Boot 4.1.

Por qué: Es el lenguaje en el que tengo más experiencia productiva y es parte del stack de Spin. Java aporta records, sealed interfaces y pattern matching, que hacen explícitos los modelos de resultado (§5.4), además de virtual threads (§5.5). Java 25 es la primera LTS en la que un virtual thread ya no queda anclado a su hilo de plataforma dentro de bloques `synchronized` (JEP 491). Eso importa porque la estrategia de concurrencia del servicio descansa en virtual threads. Spring Boot 4.1 es la línea estable vigente y la recomendada para proyectos nuevos; la línea 3.5 dejó de tener soporte OSS en junio de 2026.

Trade-off: Spring Boot 4 trae cambios incompatibles respecto a 3.x, como Jackson 3, nuevos paquetes y starters más granulares. Para un proyecto nuevo no hay costo de migración, y a cambio se parte de una base con soporte vigente. Algunas librerías requieren su módulo específico para Boot 4; por ejemplo, Resilience4j se integra con `resilience4j-spring-boot4`.

5.2 Acceso a datos: SQL explícito con JdbcClient

Decisión: `JdbcClient` de Spring con SQL escrito a mano y parámetros nombrados, en lugar de JPA/Hibernate.

Por qué: Las operaciones críticas son SQL muy específico: el UPDATE condicional sobre el estado, el filtro dinámico y la consulta de página con `LIMIT n + 1`. Con SQL explícito cada consulta es visible y explicable; no hay dirty checking, carga perezosa ni sesiones que cambien el comportamiento de forma implícita.

Trade-off: Más código de mapeo (un RowMapper) a cambio de control total.

5.3 Resiliencia ante el proveedor

Decisión: Timeouts explícitos en el cliente HTTP (conexión 1 s, lectura 3 s), circuit breaker de Resilience4j y reintento selectivo (ADR-04).

Por qué: Sin timeout, un proveedor lento retiene hilos indefinidamente y la latencia se propaga a todos los clientes. El circuit breaker deja de llamar a un proveedor degradado durante una ventana (10 s), protege a ambos lados y permite responder rápido en lugar de esperar timeouts en cadena. Se configura con ventana de 50 llamadas, mínimo 20 y umbral de 50% de fallas.

Trade-off: Un rechazo de negocio del proveedor (por ejemplo, fondos insuficientes) **no** cuenta como falla para el circuit breaker: es el proveedor funcionando correctamente. Solo cuentan timeouts, errores 5xx sin respuesta interpretable y errores de conexión.

Clasificación de respuestas del proveedor

Situación	Resultado	Estado final	¿Reintento?
HTTP 200 con APPROVED	Aprobado	EXECUTED	—
HTTP 4xx/5xx con cuerpo REJECTED	Rechazo de negocio	REJECTED	No
HTTP 4xx sin cuerpo interpretable	No ejecutado	FAILED	No
HTTP 5xx sin cuerpo interpretable	Incierto	UNKNOWN	No
Timeout de lectura	Incierto	UNKNOWN	No
Conexión rechazada o timeout de conexión	No ejecutado	FAILED	Sí, una vez
Circuit breaker abierto	No ejecutado	FAILED	No

> **Regla de oro:** si no hay certeza de que la petición no llegó al proveedor, el resultado es incierto (UNKNOWN), nunca fallido.

5.4 Resultado del proveedor como tipo cerrado

Decisión: El puerto del proveedor devuelve un resultado tipado (sealed interface con cuatro variantes) en lugar de lanzar excepciones para fallas esperadas.

Por qué: Las fallas del proveedor no son excepcionales: son parte del flujo normal y cada una tiene un tratamiento distinto. Con un tipo cerrado y un switch exhaustivo, el compilador obliga a manejar cada caso; si se agrega uno nuevo, el código que no lo contemple no compila.

5.5 Concurrencia: virtual threads

Decisión: Habilitar virtual threads de Java 25 para atender solicitudes.

Por qué: Cada solicitud pasa la mayor parte de su tiempo esperando la red del proveedor. Con hilos de plataforma, la concurrencia queda limitada por el tamaño del pool; con virtual threads, miles de solicitudes pueden esperar I/O sin consumir un hilo del sistema operativo cada una. Se obtiene la escalabilidad de un modelo reactivo manteniendo código síncrono y legible.

Trade-off: El cuello de botella pasa a ser el pool de conexiones de base de datos, razón adicional para mantener las transacciones cortas (ADR-03).

5.6 Idempotencia del API

Decisión: Header obligatorio `Idempotency-Key`, guardado junto con un hash SHA-256 del contenido de la solicitud y protegido por un constraint UNIQUE.

Por qué: El cliente también puede reintentar (por un timeout de su lado). Con la llave, el reintento devuelve el mismo resultado sin volver a llamar al proveedor. La unicidad la garantiza la base de datos, así que dos solicitudes concurrentes con la misma llave no pueden crear dos transacciones: una gana el INSERT y la otra lee el resultado existente.

Trade-off: Si la llave se reutiliza con otro contenido se responde 422; si la original sigue en curso, 409. Sin el header no hay deduplicación, y queda documentado como responsabilidad del cliente.

Nivel de aislamiento

Se usa `READ COMMITTED`, el valor por defecto de PostgreSQL. La correctitud no depende del nivel de aislamiento sino de dos operaciones atómicas por fila: el constraint UNIQUE y el UPDATE condicional sobre el estado. Un nivel `SERIALIZABLE` solo agregaría reintentos por conflictos de serialización sin proteger nada adicional.

5.7 Representación del dinero

Decisión: `BigDecimal` en Java y `NUMERIC(19,2)` en la base de datos, con escala fija de 2 decimales.

Por qué: Los tipos de punto flotante no representan con exactitud valores como 0.10, y en dinero ese error se acumula. Un monto con más de dos decimales se **rechaza**, no se redondea: redondear dinero en silencio cambia lo que el cliente pidió.

5.8 Identificadores: UUIDv7

Decisión: Identificadores UUID versión 7, generados por la aplicación.

Por qué: Los UUIDv7 están ordenados por tiempo, así que las inserciones caen al final del índice en lugar de en posiciones aleatorias, lo que mejora la localidad y reduce la fragmentación a alto volumen. Siguen siendo únicos sin coordinación entre instancias y no exponen un contador secuencial.

5.9 Errores y códigos HTTP

Decisión: Errores en formato estándar RFC 9457 (Problem Details), soportado de forma nativa por Spring.

Por qué: Un formato estándar permite a los clientes tratar todos los errores igual. Las violaciones de reglas se devuelven completas en una sola respuesta.

Trade-off: Un rechazo del proveedor responde 201 y no un 4xx: la solicitud se procesó correctamente y la transacción quedó registrada con un resultado de negocio válido, como una autorización bancaria declinada. Responder 422 o 402 también sería defendible, pero mezclaría 'tu solicitud está mal' con 'el proveedor dijo que no'.

Situación	HTTP	Se persiste	Se llama al proveedor
Solicitud mal formada	400	No	No
Regla de negocio violada	422	No	No
EXECUTED	201	Sí	Sí
REJECTED (rechazo del proveedor)	201	Sí	Sí
UNKNOWN (resultado incierto)	202	Sí	Sí
FAILED (no se ejecutó)	503	Sí	No / sin éxito
Misma llave, mismo contenido	Código original	No (replay)	No

Situación	HTTP	Se persiste	Se llama al proveedor
Misma llave, contenido distinto	422	No	No
Misma llave, original en curso	409	No	No
Idempotency_key	400	No	No

5.10 Paginación

Decisión: Paginación por `page` y `limit` (máximo 100), ordenada por fecha de creación descendente, sin total de registros: se piden `limit + 1` filas para saber si hay una página siguiente.

Por qué: Es lo que pide el reto. Calcular el total con `COUNT(*)` sobre una tabla de millones de filas es costoso en cada consulta y rara vez lo necesita el cliente; el indicador `hasNext` resuelve la navegación.

Trade-off: La paginación por desplazamiento se degrada en páginas profundas. La evolución natural es paginación por cursor (keyset).

5.11 Proveedor simulado con WireMock

Decisión: WireMock, con escenarios disparados por el número de cuenta (aprobado, fondos insuficientes, cuenta bloqueada, error sin cuerpo, lentitud, conexión cortada).

Por qué: El mismo simulador sirve para las pruebas de integración y para el ambiente en contenedores y en AWS. Simula latencias y fallas de red reales, que un stub en código no reproduce, y permite que los revisores disparen cada escenario desde Postman sin modificar nada.

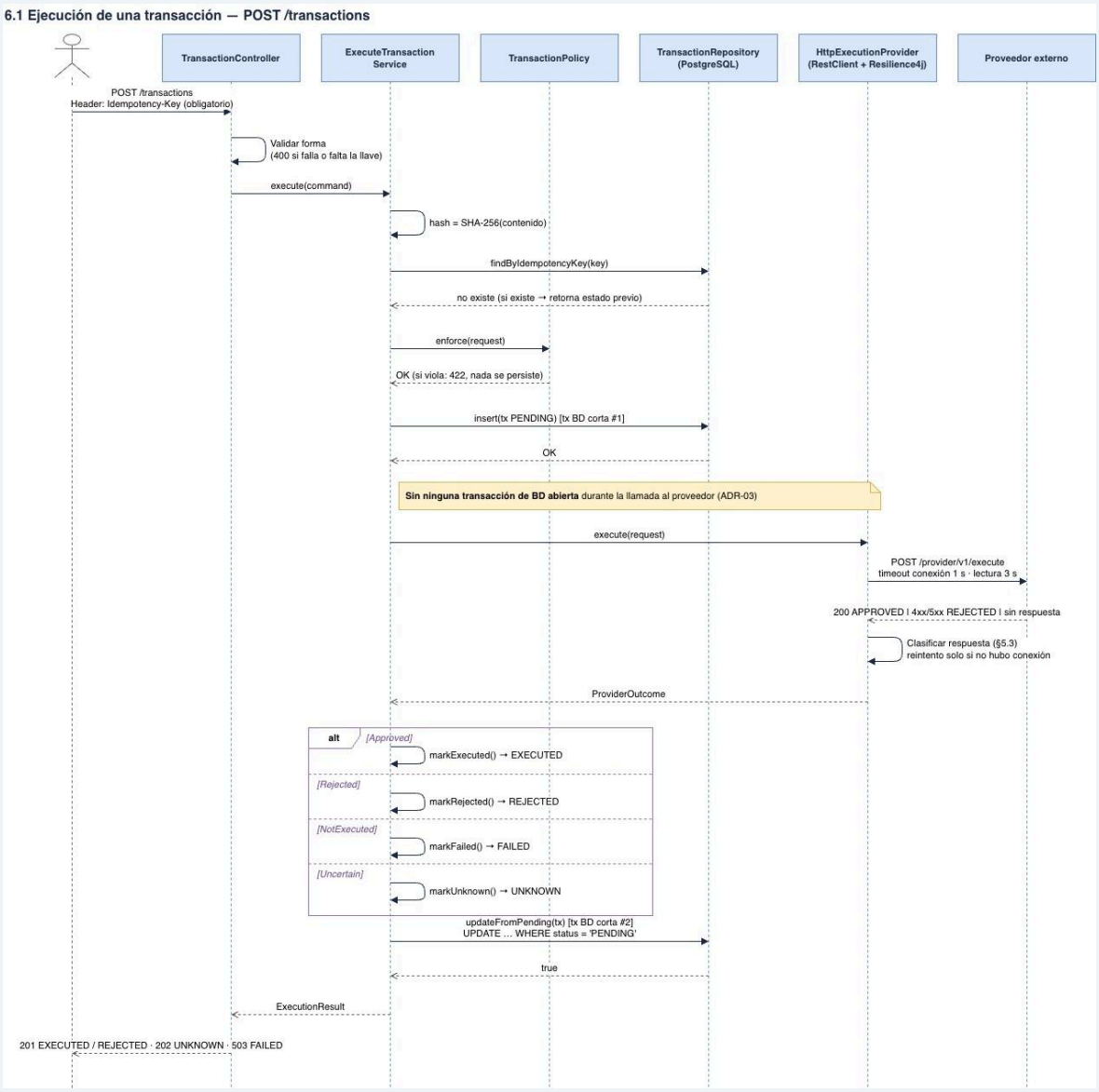
5.12 Migraciones

Decisión: Esquema versionado con changelogs de Liquibase en YAML, aplicados al arrancar el servicio.

Por qué: Liquibase hace el esquema reproducible en todos los ambientes y es la herramienta que más he usado en producción. Su versión Community incluye rollback, y los changelogs escritos con sus tipos abstractos (`createTable`, `createIndex`) son portables entre motores de base de datos.

6. Flujos principales

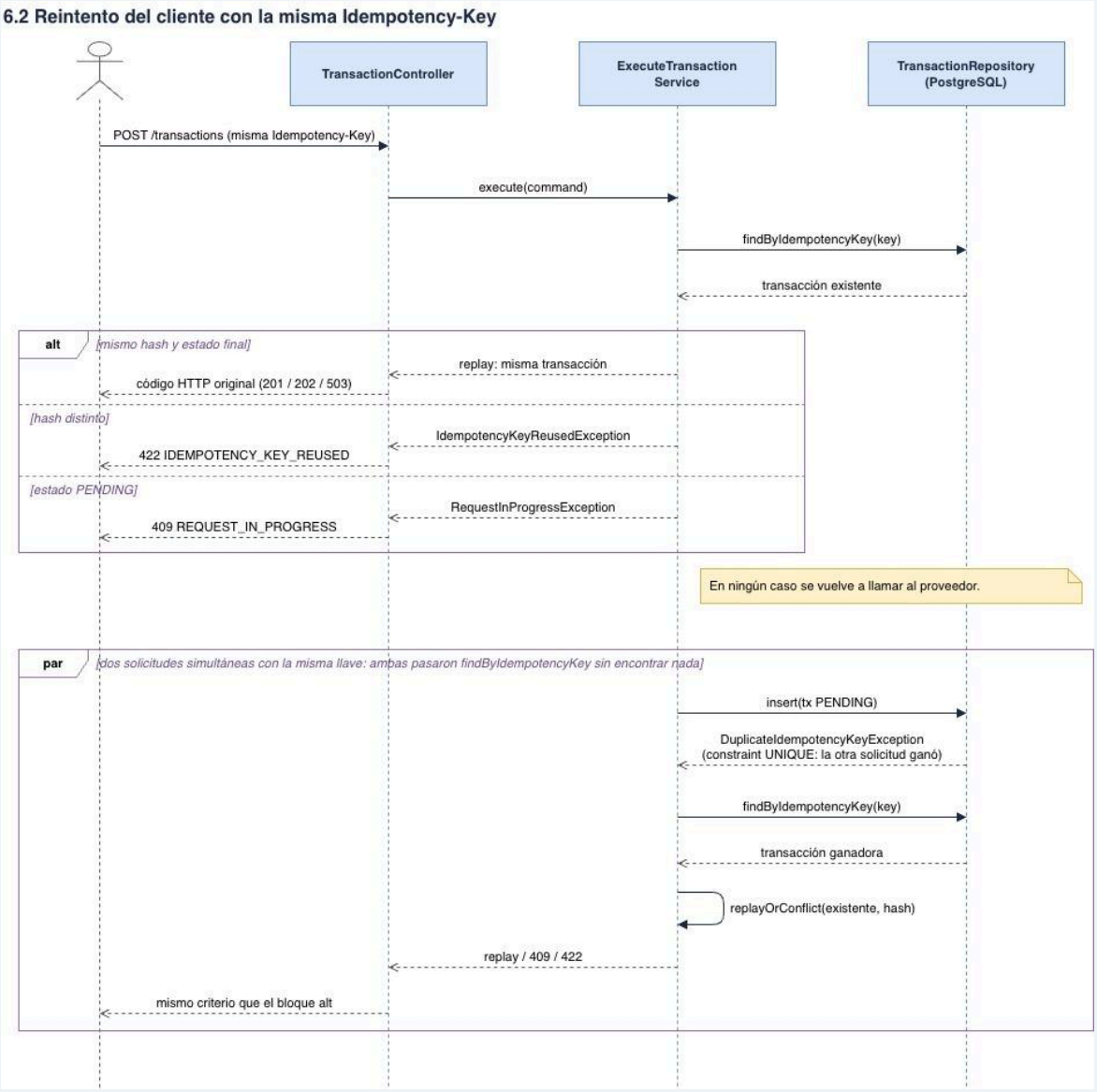
6.1 Ejecución de una transacción



1. El cliente envía la solicitud con su `Idempotency-Key`.
2. La API válida la forma de la solicitud (llave de idempotencia presente, campos obligatorios, tipos, máximo dos decimales). Si falla, responde 400.

3. Si ya existe una transacción con esa llave se responde sin continuar.
4. Se evalúan las reglas de negocio. Si hay alguna falla, se responde 422 con todas las violaciones; no se persiste nada ni se llama al proveedor.
5. Se registra la transacción en estado PENDING (transacción de base de datos corta).
6. Se llama al proveedor, **sin** ninguna transacción de base de datos abierta.
7. El resultado se clasifica (§5.3) y se aplica la transición de estado correspondiente.
8. Se actualiza el estado solo si sigue en PENDING (segunda transacción corta).
9. Se responde con la transacción y el código HTTP según su estado (§5.9).

6.2 Reintento del cliente (idempotencia)



Caso	Respuesta
La llave existe, el contenido coincide y la transacción ya tiene estado final	Se devuelve la misma transacción con su código HTTP original. No se llama al proveedor.
La llave existe pero el contenido es distinto	422 IDEMPOTENCY_KEY_REUSED.

Caso	Respuesta
La llave existe y la transacción sigue en PENDING	409 REQUEST_IN_PROGRESS. El cliente puede consultar más tarde.
Dos solicitudes simultáneas con la misma llave	Una gana el INSERT; la otra detecta el conflicto del constraint UNIQUE, lee la existente y responde como en las filas anteriores.

6.3 Estados de una transacción

Estado	Significado	¿Se movió dinero?
PENDING	Registrada; la ejecución está en curso.	Aún no se sabe.
EXECUTED	El proveedor aprobó.	Sí.
REJECTED	El proveedor la rechazó por una regla suya (por ejemplo, fondos insuficientes).	No.
FAILED	La petición nunca llegó al proveedor.	No, con certeza.
UNKNOWN	La petición pudo llegar, pero no hubo respuesta confiable.	Tal vez; requiere conciliación.

Las transiciones solo pueden salir de PENDING y cada estado final es definitivo. El dominio rechaza cualquier otra transición y la base de datos lo refuerza con el UPDATE condicional.

7. Modelo de datos

Una sola entidad, `transactions`. Los valores permitidos de tipo, estado y monto se refuerzan con constraints CHECK, de modo que la base de datos rechaza datos inválidos aunque provengan de un error en el código.

Campo	Tipo	Descripción
id	UUID (PK)	Identificador UUIDv7 generado por la aplicación.
idempotency_key	VARCHAR(100), UNIQUE, NOT NULL	Llave enviada por el cliente; obligatoria.
request_hash	CHAR(64)	SHA-256 del contenido de la solicitud, para detectar reutilización de llaves.
account_id	VARCHAR(64)	Cuenta afectada.
type	VARCHAR(10)	CREDIT o DEBIT.
amount	NUMERIC(19,2)	Monto; mayor que cero.
currency	CHAR(3)	Moneda (MXN).
description	VARCHAR(255)	Descripción opcional.
status	VARCHAR(20)	PENDING, EXECUTED, REJECTED, FAILED o UNKNOWN.
provider_transaction_id	VARCHAR(100)	Identificador asignado por el proveedor.
balance_after	NUMERIC(19,2)	Saldo reportado por el proveedor tras ejecutar.
failure_code / failure_message	VARCHAR	Motivo del rechazo o de la falla.
created_at / updated_at / executed_at	TIMESTAMPTZ	Marcas de tiempo en UTC.

7.1 Índices

Índice	Sirve a
(account_id, created_at DESC, id DESC)	Consulta principal: transacciones de una cuenta, más recientes primero.
(status, created_at DESC)	Filtro por estado y búsqueda de PENDING antiguos por parte del depurador.
(created_at DESC, id DESC)	Consulta sin filtros.
UNIQUE (idempotency_key)	Garantía de idempotencia: dos solicitudes con la misma llave no generan dos transacciones.

8. Estrategia de pruebas

Nivel	Qué valida	Herramientas
Unitarias de dominio	Cada regla en sus valores límite (1.00, 1.01, 10,000.00, 10,000.01); todas las violaciones juntas; transiciones de estado válidas e inválidas.	JUnit 5, AssertJ
Unitarias de casos de uso	Cada resultado del proveedor y su estado final; reglas fallidas que no persisten ni llaman al proveedor; idempotencia (replay, conflicto, en curso, carrera).	Dobles en memoria de los puertos
Capa web	Mapeo de cada situación a su código HTTP y formato de error.	@WebMvcTest
Integración con el proveedor	Cada fila de la tabla de clasificación (§5.3), incluidos timeouts, conexión rechazada y circuito abierto.	WireMock
Integración con la base de datos	Migraciones, unicidad de la llave, UPDATE condicional, filtros y paginación, depurador.	Testcontainers (PostgreSQL)
Extremo a extremo	Flujo completo de ejecución y consulta.	@SpringBootTest + Testcontainers + WireMock
Aceptación manual	Todos los escenarios del proveedor simulado, contra el ambiente local y AWS.	Colección de Postman

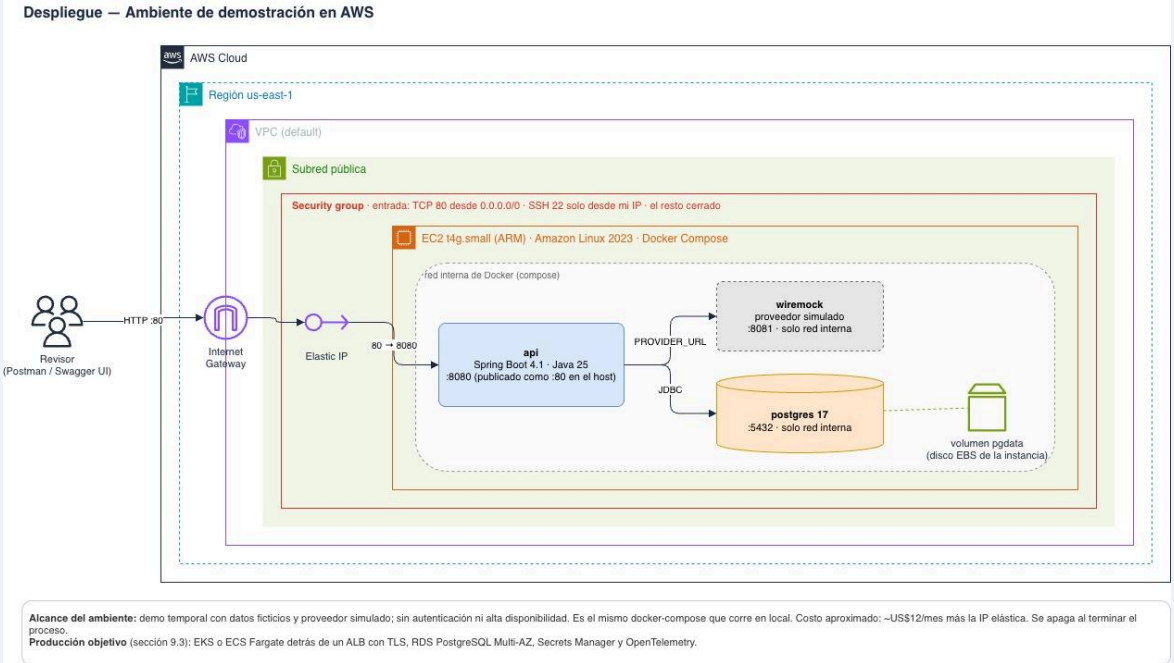
La cobertura se mide con JaCoCo. La meta es cubrir prácticamente todo el dominio y los casos de uso, y superar el 85% global.

9. Despliegue

9.1 Ambiente local

Un solo comando (`docker compose up`) levanta la API, PostgreSQL y el proveedor simulado con todos sus escenarios. La documentación interactiva del API (Swagger UI) queda disponible en el mismo servicio.

9.2 Ambiente de demostración en AWS



- Instancia EC2 t4g.small con Amazon Linux 2023, ejecutando el mismo docker-compose que en local.
- Grupo de seguridad con solo el puerto de la API abierto; PostgreSQL y el simulador quedan en la red interna de Docker.
- Sin autenticación: datos ficticios y proveedor simulado. El ambiente se apaga al terminar el proceso.
- Se entrega una colección de Postman configurada contra la URL pública.

9.3 Topología productiva objetivo

No se implementa, pero el diseño la soporta sin cambios en el núcleo:

- Contenedores en EKS o ECS Fargate, con varias instancias detrás de un balanceador con TLS.
- RDS PostgreSQL Multi-AZ; credenciales en Secrets Manager.

- Autenticación entre servicios con OAuth2 o mTLS y límites de tasa por cliente.
- Métricas y trazas con OpenTelemetry; alertas sobre el volumen de transacciones UNKNOWN y la latencia del proveedor.

10. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Doble cargo por reintentos hacia un proveedor no idempotente	Alto	Reintento solo en errores de conexión; estado UNKNOWN en lugar de reintentar (ADR-04).
Doble ejecución por reintentos del cliente	Alto	Idempotency-Key con constraint UNIQUE (§5.6).
Transacciones atascadas en PENDING por caída del proceso	Medio	Depurador periódico que las mueve a UNKNOWN (ADR-03).
Proveedor lento que agota recursos	Alto	Timeouts, circuit breaker y transacciones de BD cortas.
Crecimiento de la tabla a millones de filas	Medio	Índices alineados a consultas, sin COUNT(*); evolución a particionado y paginación por cursor.
Transacciones UNKNOWN sin resolver	Medio	Quedan identificadas y consultables; la conciliación es el siguiente paso (§11).

11. Evolución y fuera de alcance

Tema	Estado actual	Siguiente paso
Conciliación	Las transacciones inciertas se marcan como UNKNOWN.	Proceso que las resuelve contra un reporte o consulta del proveedor.
Eventos hacia otros servicios	No se publican.	Patrón outbox: registrar el evento en la misma transacción que el cambio de estado y publicarlo a Kafka mediante CDC (Debezium). Evita publicar eventos de transacciones no guardadas y viceversa.
Paginación a gran escala	Por desplazamiento.	Paginación por cursor (keyset).
Volumen histórico	Una tabla.	Particionado por fecha y archivado.
Seguridad	Sin autenticación (demo).	OAuth2, límites de tasa por cliente.
Varias instancias del depurador	El UPDATE condicional es seguro con concurrencia.	Bloqueo distribuido (ShedLock) para evitar trabajo duplicado.

12. Uso de inteligencia artificial

Se usó Claude (Anthropic) como apoyo durante el diseño y la implementación. Las decisiones de este documento fueron revisadas, ajustadas y son defendidas por el autor.

Etapa	Uso de IA	Criterio del autor
Diseño	Discusión de alternativas y redacción inicial de este documento.	[Completar: decisiones que se ajustaron o cambiaron]
Implementación	Generación de código por etapas, con revisión y preguntas de comprensión al cierre de cada una.	[Completar al terminar]
Pruebas	Propuesta de casos límite y escenarios de falla.	[Completar al terminar]

El registro detallado por etapa se encuentra en `docs/AI_USAGE.md` del repositorio.